Computers & Security 148 (2025) 104139

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

VulTR: Software vulnerability detection model based on multi-layer key feature enhancement

Haitao He [a,b], Sheng Wang [a,b,*], Yanmin Wang [a,b], Ke Liu [a,b], Lu Yu [c]

[a] School of Information Science and Engineering, Yanshan University, Qinhuangdao, China
[b] The Key Laboratory for Software Engineering of Hebei Province, Yanshan University, Qinhuangdao, China
[c] Hebei Port Group CO., LTD, China

ARTICLE INFO

Keywords:
Importance assessment
Relational connection graph
Vulnerability detection
Multidimensional convolutional model

ABSTRACT

Software vulnerabilities pose a huge threat to current network security, which continues to lead to data leaks and system damage. In order to effectively identify and patch these vulnerabilities, researchers have proposed automated detection methods based on deep learning. However, most of the existing methods only rely on single-dimensional data representation and fail to fully explore the composite characteristics of the code. Among them, the sequence embedding method fails to effectively capture the structural characteristics of the code, while the graph embedding method focuses more on the global characteristics of the overall graph structure and is still insufficient in optimizing the representation of nodes. In view of this, this paper constructs the VulTR model, which incorporates an importance assessment mechanism to strengthen the key syntax levels of the source code (from lexical elements to nodes and graph-level structures), significantly improving the importance of key vulnerability features in classification decisions. At the same time, a relationship connection diagram is constructed to describe the spatial characteristics of the correlations between functions. Experimentally verified, VulTR's F1 scores on both synthetic and real data sets exceed those of the compared models (VulDeePecker, SySeVR, Devign, VulCNN, IVDetect, and mVulPreter).

1. Introduction

With the rapid development and popularization of network technology, the cybersecurity threats faced by computer systems are increasing. Among them, software plays a core role in these systems, and its security vulnerabilities not only affect personal privacy protection and the expansion of system architecture, but may also cause serious damage to the national economy. For example, in the first quarter of 2022, the number of vulnerabilities reported in the US vulnerability database reached 8051 Cyber security vulnerability (2022a), an increase of 25 % 2022 open source security (2022b) over the same period last year. These vulnerabilities are often exploited by hackers to trigger various cybersecurity incidents such as ransomware and information leakage, seriously threatening the lives and property of citizens and national security. Therefore, improving the security and stability of software has become a major challenge at present. Among them, the detection and mining of software vulnerabilities is not only the core technology for maintaining network security, but also a key means to identify and promptly repair potential weaknesses (Lomio et al., 2022). However, with the diversification of network attacks, the difficulty of vulnerability detection and mining is also increasing.

In recent years, the introduction of artificial intelligence technology has injected new vitality into the field of software vulnerability detection. Traditionally, researchers have adopted data-driven machine learning methods to use vulnerability features predefined by experts to achieve automatic learning of vulnerable source code features. However, this method relies on complex feature engineering and expert experience, which limits its widespread application. With the development of deep learning technology (Lecun et al., 2015), source code vulnerability detection models based on deep learning have become a research hotspot because they can automatically extract vulnerability features and classify them. At present, source code vulnerability detection models based on deep learning are mainly divided into text-based models and graph-based models. Text-based models embed source code into vector space for learning by using technologies related to natural language processing. Although they have good scalability, such models usually ignore the grammatical structure of the code and fail to accommodate enough semantic information. In order to overcome this

* Corresponding author.
E-mail address: wangshengzz@stumail.ysu.edu.cn (S. Wang).

https://doi.org/10.1016/j.cose.2024.104139
Received 29 April 2024; Received in revised form 29 August 2024; Accepted 27 September 2024
Available online 29 September 2024

defect, researchers introduced the form of graph representation by analyzing the attribute graph of the code, and incorporated the syntax and semantics of the source code through the graph structure, thereby introducing a graph training model for vulnerability detection.

In graph neural networks, the node embedding stage is crucial, as it is responsible for converting the nodes in the graph into vector representations that express their features and contextual information. Since the sub-word units within the node are interdependent, accurately modeling the dependencies between these sub-words is the key to improving the quality of embedding. Although traditional embedding models (Mikolov et al., 2013; Pagliardini et al., 2017) can capture the semantic similarity between words, when processing code, they lack the understanding of structural patterns and grammatical dependencies, and often face the Out-of-Vocabulary (OOV) problem (Sennrich et al., 2015). In addition, the reasonable aggregation of these token embeddings is also very important for forming the overall vector representation of the node. Therefore, this paper uses a pre-trained model to learn relevant grammatical knowledge in the initialization stage, and enables the model to understand the dependencies between tokens through building modules.

In the graph feature extraction stage, graph neural networks are often used to obtain complex hierarchical information in the attribute graph. In order to effectively learn the representation of the code structure graph, BGNN4VD (Cao et al., 2021) uses BGNN to analyze the positive and negative sequences of nodes to capture the dependencies between source codes; Devign (Zhou et al., 2019) explores a wider range of vulnerability patterns by integrating multiple source code features. Finally, a gated graph recurrent network is used to deeply learn node embedding. However, GNN shows certain limitations when dealing with long-distance connections between non-directly adjacent nodes (Hellendoorn et al., 2019; Zhu et al., 2021). Although multi-layer GNN can be used to learn the global information of the graph, this approach may lead to the problem of over-smoothing (Alon et al., 2020; Li et al., 2018a), where nodes with different labels produce similar embedding results, thereby affecting the overall performance of the model (Pei et al., 2020).

Although existing GNN models show powerful capabilities in processing graph data, they often rely too much on global features and ignore the characterization of local information, especially the importance of individual nodes. In order to strengthen the importance of nodes in the network, TreeCen (Hu et al., 2022) uses node centrality to convert the AST graph into a fixed-length vector that retains structural information. Although this method highlights the strategic position of nodes, it only considers one centrality and has a single feature angle; VulCNN (Wu et al., 2022) focuses on the extraction of local features. It analyzes the centrality of different nodes in the network graph, takes it as a three-channel input of the image, and uses CNN instead of GNN to process it. Although this method combines three centralities, the processing is relatively rough. Based on this, this study conducts experiments by combining three different social network analysis angles - involving core nodes in the graph, connection relationships between nodes, and their distances - in order to find a better combination of centralities. In addition, current deep learning-based methods (Y Li et al., 2021; Zou et al., 2022) have limitations in processing inter-function relationship features and may lose key information. Given the reuse of function code and the consistency of class implementation, similar functional patterns may appear in multiple functions. Therefore, correctly understanding the similarities between these functions is crucial to capturing inter-function relationships.

Based on the issues mentioned above, this paper proposes a PDG (Yamaguchi et al., 2014) analysis method based on code statement nodes, which processes source code through multi-level importance evaluations from the token, node, to graph levels, aiming to reduce the similarity of the final embedding results by enhancing key features at these levels. Firstly, in the token-level evaluation, the importance of tokens is assessed using a node embedding model optimized with pre-trained weights. Secondly, the node-level evaluation treats the program dependency graph as a complex social network, identifying key nodes through centrality analysis. Lastly, the code attribute graph-level evaluation enhances the analysis of crucial parts of the graph structure using k-GCNs graph convolutional networks combined with a global attention mechanism, thereby improving the understanding of features within functions. Additionally, to explore and utilize potential similarities between functions, a clustering algorithm is introduced to construct a new type of functional relationship graph. Finally, by integrating and analyzing features within and outside of functions, training and prediction of vulnerability data are achieved.

The main contributions of this paper are as follows:

1. A k-GCNs model incorporating a triple importance mechanism (token-level, node-level, and graph-level) is proposed to efficiently capture the feature representation of code dependency graphs, enhancing key vulnerability features.
2. Considering the relationships between different entities, a relationship connection Graph based on relational space semantics is proposed to capture the intrinsic connections between functions.
3. A multi-feature fusion vulnerability detection model named VulTR is proposed and has been extensively evaluated on three public datasets. Experimental results indicate that VulTR performs better than existing slice-level and function-level models.

The subsequent chapters of this paper will be developed according to the following structure: Section 2 introduces the relevant research results of existing vulnerability detection technologies; Section 3 details the specific implementation method of the VulTR framework proposed in this paper; Section 4 verifies this method through comparative experiments and ablation experiments. The remaining chapters discuss future research directions and the overall conclusion.

The open-source code will be released at https://github.com/1597879264/VulTR.

2. Related work

2.1. Traditional vulnerability detection techniques

Traditional vulnerability detection technology is mainly divided into two methods: dynamic acquisition and static detection. Dynamic analysis (Liu et al., 2012; Newsome et al., 2005) requires running the program to retrieve the vulnerabilities discovered during the execution path. It is mainly divided into fuzz testing, symbolic execution and other methods. Since dynamic analysis requires configuring different environments for the running of different programs, for large projects, it is difficult for the analyzer to cover the entire code space, which places high requirements on the system. Static vulnerability (Ayewah et al., 2007; Johnson et al., 2013; Smith et al., 2015) mining refers to analyzing the target program without running the source code, including the analysis of source code and binary syntax, semantics, and other information. It detects vulnerabilities by converting them into various intermediate representations. Both dynamic and static technologies can provide effective guarantees for the security of software systems, but considering the actual operating environment and scope of use, most researchers prefer to use static analysis methods to mine software vulnerabilities before software application to reduce the risk of online various threats that may be faced in the future.

Static vulnerability detection can be divided into two methods based on code similarity and pattern-based. Similarity-based detection is mainly used to detect code cloning errors (Jang et al., 2012; Kim et al., 2017; Pham et al., 2010). This method uses manually selected code fragments as vulnerability features and determines by calculating the similarity between the target code segment and the vulnerable code segment. Are there any loopholes? Pattern-based methods are divided into rule-based and machine-learning-based methods. Rule-based methods mostly rely on expert experience to define relevant

vulnerability characteristics and are affected by the subjectivity of manual intervention. Common static analysis tools mainly include Clang Static Analyzer (2020a),Flawfinder (2020b),Checkmarx (2020c), etc.

2.2. Vulnerability detection model based on deep learning

With the rapid development of deep learning technology, learning features from source code to automatically detect vulnerabilities has become the focus of security experts. Existing vulnerability detection models based on deep learning are mainly divided into function level (Chakraborty et al., 2021; Duan et al., 2019; Neamtiu et al., 2005) and slice level (Z Li et al., 2021; Zou et al., 2019) according to the size of sample granularity. Function-level vulnerability detection models treat functions of source code as processing units. For example, Feng et al. (H et al., 2020) obtain the sequence representation by traversing the Abstract Syntax Tree (AST) (Neamtiu et al., 2005) of the code and inputs it into the BGRU model for training and prediction. VulCNN (Wu et al., 2022) converts the source code into an image representation that fuses three-channel information, and uses a convolutional model to classify source code vulnerabilities. In order to further improve the precision and accuracy of vulnerability detection, researchers have proposed a slice-level fine-grained vulnerability detection method, by extracting functions from the "interest points" of the code into different code snippets to more accurately determine where the vulnerability is located. For example, VulDeePecker (Li et al., 2018) is the first system to use a deep learning model to detect the feasibility of vulnerabilities at the slice level. It collects program slices of sensitive APIs in code segments through a slicing algorithm, vectorizes them, and inputs them into the model for detection; SySeVR (Zhen et al., 2021)uses multiple dependencies such as data dependencies, control dependencies and call dependencies are used to construct slices to comprehensively capture the structural and behavioral characteristics of the source code; Devign (Zhou et al., 2019) processes the source code into four different baseline types of vulnerability slices and constructs AST, Control Flow Graph (CFG), and Data Flow Graph (DFG). This allows the slices to accommodate more semantic and syntactic information, thereby improving detection efficiency.

Vulnerability detection models based on deep learning usually use two different representations to represent source code, which are textual representation and graphical representation (Cao et al., 2021; Zhou et al., 2019; Y et al., 2023).The vulnerability detection model based on the text model utilizes technologies related to natural language processing, treating the source code as different mark sequences, and then embedding them into vector space for learning to achieve the vulnerability detection process. For example, VulDeePecker (Li et al., 2018) reduces the detection scope of function vulnerabilities by obtaining relevant program slice gadgets, and passing it into the BiLSTM model for source code vulnerability detection; TokenCNN (Russell et al., 2018) extracted the code token sequence as input and used the convolutional neural network to extract features from the text. These features are then passed to the MLP layer for vulnerability classification. As code vulnerabilities diversify, researchers realize that a deeper understanding of the semantic information in code is necessary to improve detection accuracy. Therefore, researchers introduced graph representation methods by analyzing different types of syntax trees and semantic graphs of codes. This method represents the source code as graph-structure nodes and the dependencies as graph edges. These elements fully contain the syntax and semantic information of the code and introduce a new feature acquisition method for vulnerability detection. For example, the Devign analyzes AST to obtain a CFG and DFG, and uses a convolution module to extract useful features from rich node representations by constructing a joint graph for graph-level classification; DeepWukong (Xiao et al., 2021) uses the slicing algorithm to obtain the subgraph of Program Dependence Graph (PDG), and extracts the structure (edges of XFG) and content (nodes of XFG) information of code processes based on XFG, and finally performs source code-oriented vulnerability detection through the GNN model. According to related research (Velickovic et al., 2018), graph-level classification can incorporate more semantic and syntactic information, enhancing the efficiency of vulnerability detection. This approach is superior to models based solely on textual content.

Based on the aforementioned research activities, this paper selects VulDeePecker, SySeVR, Devign, VulCNN, IVDetect, and mVulPreter as models for comparative analysis. Table 1 delineates the pertinent characteristics of these models for evaluation.

3. System architecture

3.1. Overview

The model proposed in this paper comprehensively considers the semantic and grammatical information of the code, and uses the multi-importance mechanism and relational graph encoding to extract embedding and relational features. Specifically, the VulTR model consists of four stages, as shown in Fig. 1.

1) Initialization stage (PDG construction): The source code is normalized to remove irrelevant semantic information and converted into a PDG, which serves as a key research object for vulnerability feature analysis;
2) Feature extraction stage (multiple importance evaluation): By evaluating the importance of elements at the PDG token, node, and graph levels, the k-GCNs model is used to optimize the code feature extraction method and strengthen the identification and representation of key vulnerability features;
3) Relationship mapping stage (relationship connection graph encoding): After obtaining the intra-function features, the relationship connection graph between functions is constructed based on the similarity relationship to provide support for identifying cross-function vulnerability patterns;
4) Vulnerability identification stage (vulnerability detection and location): Comprehensive intra-function and inter-function features are used to accurately locate and highlight the vulnerability statements, thus completing the entire vulnerability detection and location process.

3.2. Data preprocessing

3.2.1. Code normalization

The VulTR model is a method for detecting function-level vulnerabilities. Since different researchers may use different naming methods for variables and functions, it is inevitable that information irrelevant to the semantics of the function code is added, affecting the training and prediction of the model. To this end, the preprocessing stage needs to normalize and standardize the data, and unify the code writing format. Fig. 2 illustrates the process of abstracting and normalizing the source code of the "CWE121_Stack_Based_Buffer_Overflow__CWE129_fgets_45_bad" function (2024):

Step 1: Use regular expressions to remove non-ASCII characters and comment characters that are irrelevant to code semantics;

Step 2: Modify custom variable names. Treat the code keywords defined in C11 and C++17, as well as parameters in the main function, as constants, and name the remaining variables in a uniform and standardized format 'VAR{number}', such as VAR1.

Step 3: Modify the custom function name. Except for the main function, make other function names have the same specification "FUN {number}", such as FUN1.

3.2.2. PDG construction

PDG optimizes the accuracy of vulnerability detection, refines the analysis scope from file level to function level, and extracts and represents vulnerable functions in a graphical way. In the process of building PDG, the first step is to use AST to analyze the source code in detail.

Table 1
Overview of comparative features of vulnerability detection models.

Model	DataType	Node Embedding	Detection Level	Classifier Model	Research Interest	Dataset
VulDeePecker	Source codes	Word2Vec	Slice	Bi-LSTM	Introduces the concept of code gadget , extracts slices for sensitive APIs/library	SARD,NVD
SySeVR	Source codes	Word2Vec	Slice	DBN,CNN, etc	Generate code slices and extract the control dependency and data dependency features of the code	SARD,NVD
VulCNN	Image	Sent2Vec	Function	CNN	Convert source code into a three-channel image representation, and extract features based on three centrality principles from visualized code	SARD,NVD
Devign	CPG	Word2Vec	Function	GGNN	Integrate abstract syntax trees, source code sequences, and their control flow and data flow graphs to explore complex vulnerability patterns	FFMPeg,Qemu
IVDetect	PDG sub-graphs, statements, dependencies	Sent2Vec	Function	Bi-GRU	Analyze data and control dependencies, focusing on statements and other contexts, and extract graph representations using GRU	BigVul,FFMPeg, Qemu,Reveal
mVulPreter	PDG	GloVe	Function	GGNN	A multi-granularity detector combining function-level and slice-level granularity, introducing attention mechanisms for graph representation of source code	BigVul

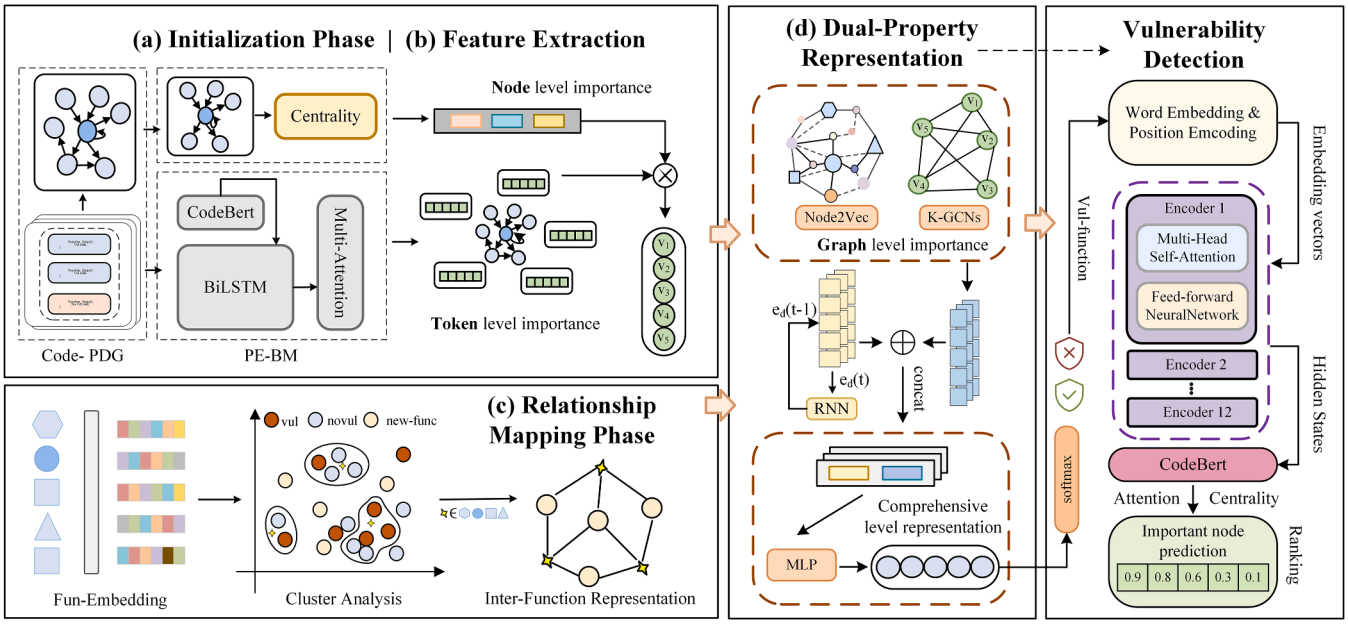

Fig. 1. VulTR Vulnerability Detection Model: (a) Initialization constructs the PDG; (b) Feature Extraction assesses sub-word, node, and graph elements for feature enhancement; (c) Relationship Mapping outlines the RCG construction; (d) Vulnerability detection through dual-property analysis.

Then, CFG and DFG are established to further express the control and data dependencies between functions to form the final graphical representation. This paper uses the open-source C/C++ static code analysis tool Joern (Okun et al., 2013) to perform this process. It can directly extract PDGs of different functions from the code file without compiling the code. Fig. 3 shows the construction result of the PDG for the source code shown in Fig. 2.

3.3. Triple importance mechanism

3.3.1. Token level importance

When graph nodes are embedded, a token-based importance evaluation mechanism is used within the nodes to obtain a more accurate semantic vector representation of each statement node. Specifically, after obtaining the static representation of the PDG of the code, the node list of the function is obtained by traversing the graph representation, and the function is split into a set of statements $S = \{s_1, s_2, s_3, \cdots, s_n\}$, where each s_i corresponds to node v_i in PDG. Traditional graph models rely on using Word2Vec (Mikolov et al., 2013) or Sent2Vec (Pagliardini et al., 2017) to build the representation of code nodes; these methods often ignore the structural and grammatical information of the code. To deeply explore the code features, this paper built the PE-BM (Pre-trained Embeddings-BiLSTM with Multi-head Attention) module, as shown in Fig 1. This module aims to give each statement node a more precise semantic vector representation. By building a Token-based importance evaluation mechanism, the PE-BM module significantly improves the depth and accuracy of semantic capture.

In the data processing stage, the model adopts a sequence padding strategy to standardize the number of tokens in each sequence in response to differences in sequence length. To solve the out-of-vocabulary (OOV) problem in the code embedding process, this layer applies the byte pair encoding (BPE) (Sennrich et al., 2015) method on the CodeSearchNet corpus. BPE reduces the vocabulary of function naming by decomposing rare words into meaningful sub-word units and divides uncommon function names into more fine-grained representation units. In this way, BPE improves the quality and expressiveness of node embedding. In addition, to further optimize the performance of the embedding layer, the pre-trained CodeBERT model is used to initialize the embedding layer weights after word segmentation. CodeBERT (Feng et al., 2020) is a model specially designed for the interaction between

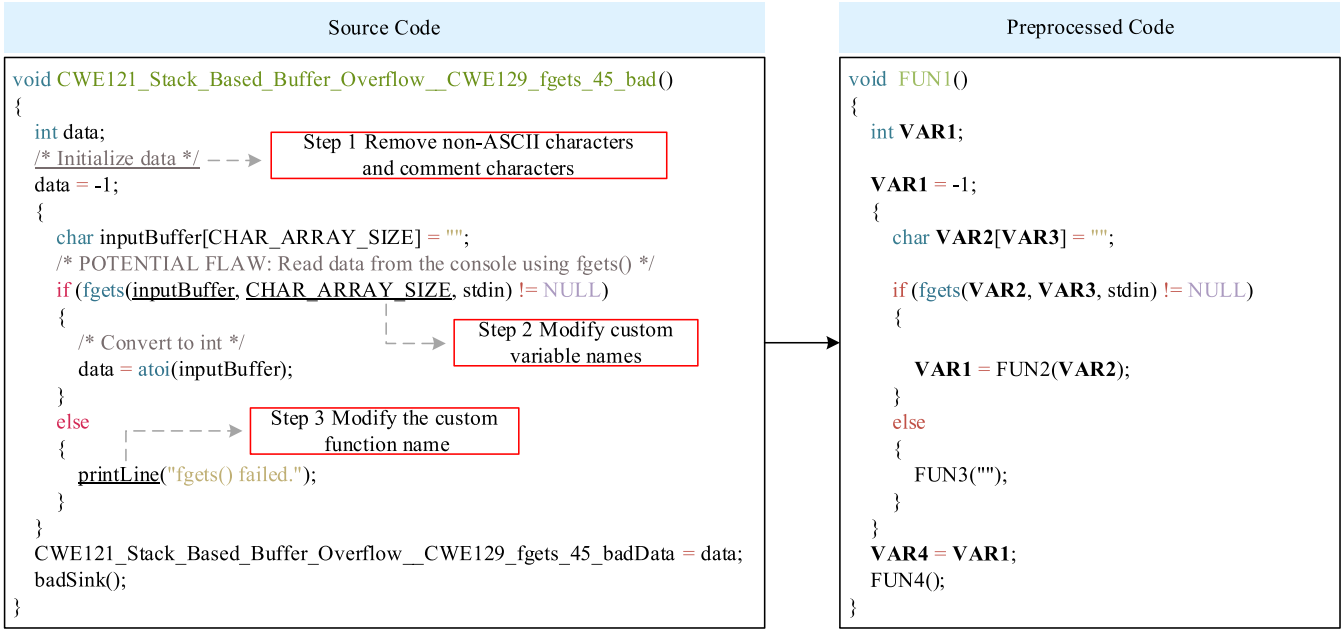

Fig. 2. The code normalization of CVE121.

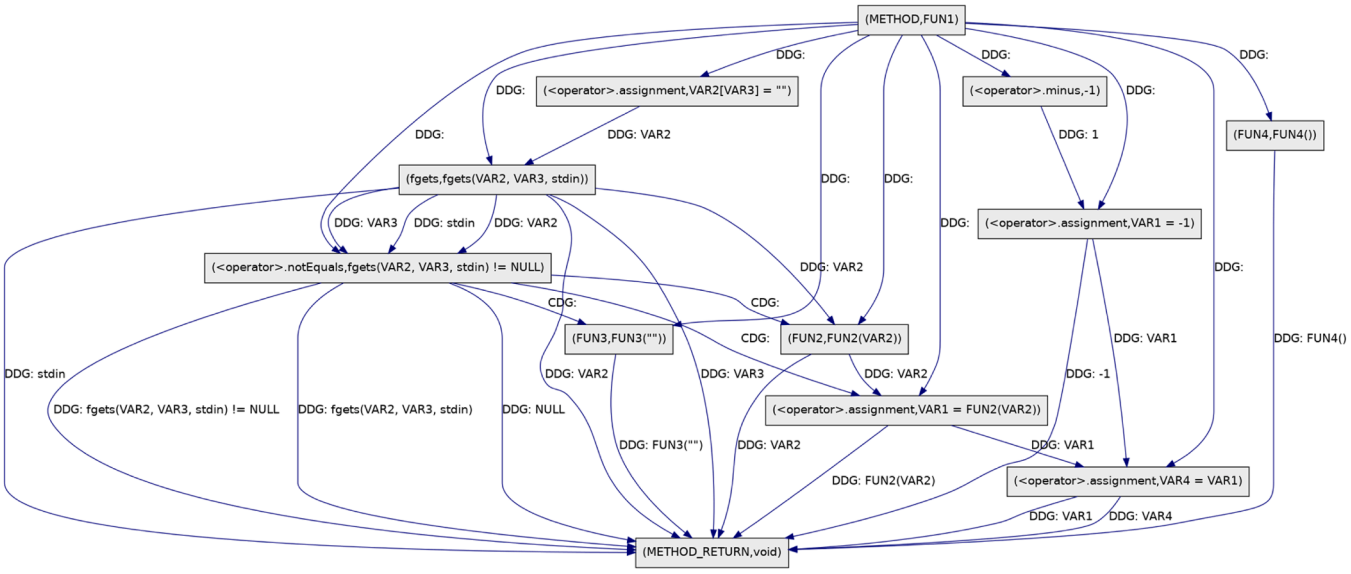

Fig. 3. The vulnerability dot of the example vulnerability source code in Fig. 2.

programming languages and natural languages. Through pre-training, it can optimize the semantic understanding between source code and natural language. It effectively identifies and encodes key elements and grammatical roles in programming languages, such as keywords, operators, and identifiers, thereby improving the model's understanding of syntax and structure.

Formally, the segmented sentence node v_i contains n tokens. After the embedding layer, the vector representation is input to BiLSTM (Hochreiter et al., 1997) to further parse the tag sequence and generate the semantic matrix $P_i = \{p_{i1}, p_{i2}, \ldots, p_{in}\} \in R^{n*m}$ of the node v_i (m is the embedding dimension).$p_{ij} \in R^{n*m}$ represents the embedding vector of the jth token in the node. PE-BM effectively captures the contextual dependencies of tokens and the grammatical structure in the code through the bidirectional structure of BiLSTM, which is particularly important for revealing dependencies such as the long distance between variable definition and use. By analyzing the relationship between the previous and next elements, the overall logic of the code is enhanced — a feature ignored by Word2Vec and Sent2Vec.

At the back end of the model, node matrix P_i is input into a stacked structure consisting of Transformer encoder blocks to calculate the semantic vector V_i corresponding to each sentence node v_i. Each encoder block consists of a bidirectional multi-head self-attention layer (Devlin et al., 2018) and a fully connected layer. By splitting the feature vectors Q, K, and V into h independent "heads", the model simultaneously pays attention to multiple key tokens in the sequence when forming the node vector. This multi-head processing configuration allows each "head" to analyze the input sequence in parallel, capturing different information dimensions and comprehensively analyzing tokens from multiple perspectives. After processing all self-attention "heads", the results are merged and linearly transformed by the weight matrix W^O. By connecting multiple such encoder blocks in series, the feature vector of the node is finally generated. The process of calculating the node feature

vector by the PE-BM module is shown below.

$$P_i = BiLSTM(PreEmbedding(tokenIds)) \tag{1}$$

$$Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2}$$

$$MultiHead(Q,K,V) = Concat(head_1,\ldots,head_h)W^O \tag{3}$$

$$\text{head}_\text{i} = Attention\left(QW_i^Q, KW_i^K, VW_i^V\right) \tag{4}$$

$$V_\text{i} = flatten(MultiHead(P_i,P_i,P_i)) \tag{5}$$

3.3.2. Node level importance

PDG offers a powerful perspective for analyzing code structure and behavior, yet individual graph analysis often fails to fully reveal the complex dependencies between nodes. Unlike traditional methods that treat all nodes equally using MLPs, VulCNN considers each line of code as a node and establishes connections through control and data flows, similar to interpersonal interactions in social networks. In this "social network," the importance of nodes is determined by their connections to others, with more critical nodes in the program structure contributing more significantly to program semantics. VulCNN uses three centrality metrics to convert code into a graph for analysis, but it does not adequately consider the differences between combinations of these centrality metrics. To address this issue, this paper evaluates the impact of different node centrality on distinguishing node importance through experiments and selects an optimal combination of social network centrality to quantify node importance, involving three aspects: core nodes, connections between nodes, and distances between nodes. Specifically, this study proposes a new method for assessing node importance that enhances performance by better strengthening node features and integrating multiple centrality perspectives, as detailed in Algorithm 1.

The degree of centrality based on the core node determines its criticality in the network by counting the number of direct connections $|V|-1$ of the node. where $deg(v_i)$ is the degree of node v_i.

$$C_D(v_i) = \frac{deg(v_i)}{|V|-1} \tag{6}$$

Betweenness centrality based on the distance between nodes is used to evaluate the frequency $\sigma_{st}(v)/\sigma_{st}$ of node v as an intermediary on the shortest path st.

$$C_B(v_i) = \frac{2}{(n-1)(n-2)}\sum_{s\neq v\neq t}\frac{\sigma_{st}(v)}{\sigma_{st}} \tag{7}$$

Eigenvector centrality enhances the concept of degree centrality by measuring not only a node's own connections but also the importance of its adjacent nodes, where A_{v_iu} is the adjacency matrix of node v_i.

$$C_E(v_i) = \frac{1}{\lambda}\sum_{u\in V}A_{v_iu}C_E(u) \tag{8}$$

Algorithm 1 describes the node-strengthening process in detail. After vectorizing all sentence nodes, the obtained PDG is used to perform social network analysis to obtain the important node features of all sentence nodes in the graph. First, the degree centrality and betweenness centrality of the nodes are calculated using the program dependency graph. These two indicators focus on the connections between nodes and ignore the directionality of the connection. For eigenvector centrality, it considers not only the direct connections of a node but also the importance of its adjacent nodes. After calculating these three types of centrality vectors, as per Algorithm 1, they are considered as feature information with three channels. For each channel, global average pooling (GAP) compresses the spatial information into a single scalar, reflecting the global features of the entire channel. Subsequently, this channel description undergoes transformation through two fully connected layers, which compress and expand it respectively, generating the importance weights for each channel. Node features are then extracted by multiplying with the original feature graph, resulting in the

Algorithm 1
Enhancing the node features.

Algorithm 1: Enhancing the node features

Input: CodeFunction
Output: NodeVectors
mF ← *CodePreprocessing(F)*
$g^{PDG}(V,E)$ ← *extract_PDG (mF)*
line=m['label']
vector ← *NodeEmbedding (line)*
S={s1,s2,...,sn} ← *ObtainLines*($g^{PDG}(V,E)$)
for *each s ∈S* ***do***
  TokenList ← *Tokenizer(s)*
  TokenEmbedding ← *CodeEmbedding(TokenList)*
  P ← *BiLSTM(Embedding)*
  Vector ← *MultiHead(P)*
  $C_D(v), C_B(v), C_E(v)$ ← *Centrality*($g^{PDG}(s,e)$)
  DegCenFeture.add($C_D(v)$**Vector)*
  BetCenFeature.add($C_D(v)$**Vector)*
  EigenVectorFeature.add($C_E(v)$**Vector)*
end
VectorArrays.add(DegCenFeture,BetCenFeature,EigenVectorFeature)
WeightValue ← *GAP(VectorArrays)*
NodeVector ← *MlpAndConv (VectorArrays* WeightValue)*
return *V′*

final graph semantic centrality sequence. This step can be represented as follows:

$$V'(v_i) = Conv\left[\sigma_{1,2}(AvgPool(C_D(v_i), C_C(v_i), C_K(v_i)))\right] \tag{9}$$

3.3.3. Graph level importance

This paper proposes a k-GCNs model combined with a triple importance mechanism to perform representation learning on PDG graphs. For each graph, node embedding is achieved by considering token-level and node-level importance. Furthermore, the graph data structure is represented as $g^{PDG}(V,E)$, where V represents the node vector and E represents the matrix of edges. During the feature acquisition process, k-GCNs can generate different numbers of convolutional pooling modules to learn the hidden states of nodes in each layer according to different data sets. By stacking convolutional layers and pooling layers k times, the model can gradually extract deeper features and achieve effective representation and classification of complex data.

Specifically, in the feature propagation stage of the model, the convolutional layer uses the weight matrix W to linearly transform the features of the current node v and its adjacent nodes. The hidden state of node v can be calculated in the following way:

$$\boldsymbol{f}^{l}(\boldsymbol{v}) = \sigma\left(\boldsymbol{f}^{(l-1)}(\boldsymbol{v})\cdot W^{l}_{CP1} + \sum_{m\in N(v)} \boldsymbol{f}^{(l-1)}(m)\cdot W^{l}_{CP2}\right) \tag{10}$$

In the formula, the weight matrices W^{l}_{CP1} and W^{l}_{CP2} of layer l will act on the output of layer $l-1$ to generate the features of the current layer. $\boldsymbol{f}^{(l-1)}(\boldsymbol{v})\cdot W^{l}_{CP1}$ represents the transformation of the current node's characteristics; $\sum_{m\in N(v)}\boldsymbol{f}^{(l-1)}(\boldsymbol{m})\cdot W^{l}_{CP2}$ represents that for each neighbor $m \in N(v)$ of node v, the characteristics at the $l-1$ layer are summed to aggregate the information of neighbor nodes. Finally, by adding the two features and using the activation function σ to perform a nonlinear transformation on the result, the output feature $\boldsymbol{f}^{l}(\boldsymbol{v})$ of the node in the l layer is obtained. To reduce the data dimension and retain important features, the pooling ratio is applied in the convolutional pooling layer to reduce the size of the original graph, so that the number of nodes is changed from $N \rightarrow KN$, reducing the computational complexity of the graph model. For the graph node feature matrix $\boldsymbol{F} = \{\boldsymbol{f}(0), \boldsymbol{f}(1), \ldots\ldots, \boldsymbol{f}(\boldsymbol{N})\}$, first obtain its projection vector, then map and reduce the dimensionality of the node features to achieve a more compact and discriminative representation. After obtaining the index values of k nodes through the Top-k (Cangea et al., 2018) algorithm, the input tensor F is element-by-element multiplied by the projection vector after the action of the tanh activation function using broadcast element multiplication to enhance the interaction between features. ability. Finally, the converted adjacency matrix M′ is obtained through the index.

$$i = top - k\left(\frac{\boldsymbol{Fn}}{\|\ n\ \|}, k\right) \tag{11}$$

$$\boldsymbol{F}' = (\boldsymbol{F}\odot \tanh)_i \quad M' = M_{i,i} \tag{12}$$

The graph node aggregation function, as the implementation of the graph-level importance mechanism, has a great impact on the performance of the model (Xu et al., 2018; Hamilton et al., 2017). After completing the graph convolution and graph pooling operations at each layer, k-GCNs uses the Soft Attention Aggregation Layer as a graph-level importance mechanism to weight and aggregate the features of the nodes, generating a global feature vector representing the entire graph. Soft attention aggregation (Li et al., 2019) is a key feature in GAT. It dynamically determines the influence of each neighboring node on the current node through the attention mechanism and can automatically learn and adjust the weights of different input elements. Specifically, for each node v_i, a linear layer is used to obtain the weight value of each node, and by adding all weighted node feature vectors, the representation $\boldsymbol{F}_{PDG}$ of the entire graph is obtained.

$$\boldsymbol{F}^{(l)}_{PDG} = \sum_{i=1}^{N^{(l)}} \mathrm{softmax}\left(\mathrm{Linear}\left(\boldsymbol{F}^{(l)}_{i}\right)\right)\cdot \boldsymbol{F}^{(l)}_{i} \tag{13}$$

In summary, through multiple importance evaluations, the model can better focus on the key features of the code attribute graph. Among them, token-level and node-level centrality focus on the importance of a single node, reflecting the relative status and influence of each node in the entire network. When the GAT graph-level attention mechanism is introduced, the focus shifts to how to adjust the representation of nodes based on the interactions between nodes and the entire graph structure. This combination can be regarded as a fusion of "bottom-up" prior knowledge and "top-down" learning mechanism, which helps the model understand and express the characteristics of graph data more comprehensively and deeply.

3.4. Relationship connection graph encoding

Since the current models are mostly discussed based on a single function, they do not involve the relationship characteristics between different functions. Therefore, this paper proposes a relationship structure RCG, which integrates the relationship features between functions, as shown in Fig. 1 Enhanced Relationship Graph stage. In traditional deep learning models, complex semantic learning of a single function often fails to effectively reveal potential vulnerabilities; while interactions between functions, such as code reuse or functional similarity, may hide important vulnerability clues. Although individual statements within a function may not convey complete semantic information independently, through feature embedding, these statements can carry enough information to allow the model to evaluate their similarity or relatedness to other statements or functions, thereby revealing interrelationships between functions.

Step 1. First, by combining improved graph embedding methods with topological features, generate the graph semantic centrality sequence of the PDG, as shown in Fig. 4, step a.

Step 2. After embedding function-level objects in the data set, clustering is used to distinguish different functional modes in the data set, including vulnerability modes and normal function modes. RCG uses the K-Means (Lloyd, 1982) algorithm to cluster functions, group functions with similar logical functions and represent them with a small number of centroids, thereby effectively reducing the number of nodes and simplifying the graph structure, as shown in step b in Fig. 4. For function embedding, this paper uses CodeBert technology to obtain features and leverages its powerful ability to capture contextual relevance in code functions. In view of the high dimensionality of the embedding vector generated by CodeBert, embedding words directly will increase the complexity of model processing. Therefore, this paper adopts the strategy of vector embedding in function units. After embedding, this layer defines a fully connected layer to adjust the dimensions of the embedding vector to match the dimension requirements of the semantic center sequence of the graph to facilitate the calculation of similarity weights.

Step 3. The construction of the RCG describes the relationships between functions. Refer to Algorithm 2 for details on the specific process. In analyzing a given function, the RCG maps the connections between internal statements and their functional patterns, articulating the relationships between functions. In this process, each function statement is considered as a node, with centroids generated by clustering serving as connection points. Connections between nodes are established by calculating the Euclidean distance to measure the similarity between statement nodes and cluster centers, which then serves as the weight for the edges. Finally, the connections between multiple statements and functional patterns are abstracted into relationships between functions and functional patterns. Notably, when multiple functions share statements highly similar to a specific vulnerability cluster, these functions

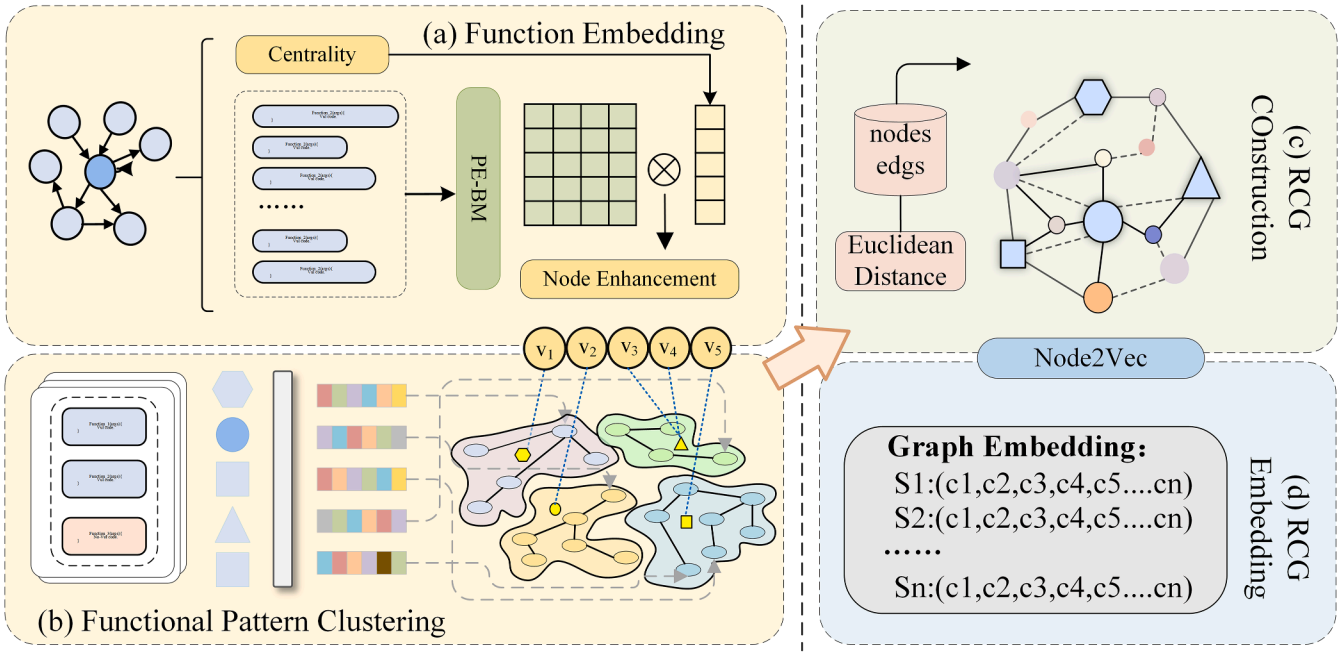

Fig. 4. The construction process of the RCG: (a) Embedding Functions via PDG for Semantic Analysis; (b) Clustering Functional Patterns Using K-Means for Data Segmentation; (c) Constructing RCG to Map Function Similarities; (d) Embedding RCG into Low-dimensional Space for Enhanced Analysis.

Algorithm 2
Constructing a relationship connection graph.

Algorithm 2: Constructing a Relationship Connection Graph

Input: CodeFunctions
Output: RCG
processedCode ← *PreprocessCode(CodeFunctions)*
FunctionVectors ← *EmbedFunctions(processedCode)*
KmeansModel ← *TrainMiniBatchKMeans(functionVectors)*
funcIndex ← *0*
for *vec* ∈ *FunctionVectors* ***do***
  closestCluster ← KmeansModel.predict(vec)
  Distance ← CalculateSimilarity(closestCluster, vec)
  V[file.name].label ← *0*;
  if *nodeWeights or useDistance* ***then***
    if *useDistance* ***then***
      weight ← *10000 / (Distancee + 1)*
      edges.append([funcIndex,closestCluster,weight])
  end
  funcIndex+= 1
end
graph ← *CreateDirectedGraph(edges)*
node2VecModel ← *InitializeNode2Vec(graph)*
nodes ←*prepareWalks(node2VecModel)*
for *node* ∈ *nodes* ***do***
  nodeId ← *node[0]*
  nodeVec ← *node[1:]*
  F_{RCG}*.append(nodeVec)*
end
return F_{RCG};

may exhibit similar vulnerability patterns, reflecting a characteristic relationship between functions. The construction results are shown in Fig. 4, step c. RCG is ultimately defined as the structure G (F, S, W), where F is the set of function nodes, S is the set of statement nodes, and W is the set of weights, specifying the weight of connections between functions and statements.

Step 4. To effectively extract the structure and content information of the RCG, graph embedding technology is applied to convert the graph structure into a low-dimensional vector. This technology helps to automatically learn the representations of weighted edges and function nodes, enhancing computational efficiency. Specifically, this study utilizes the Node2Vec algorithm for embedding, which combines deep learning with random walks. By simulating the traversal and path choices between nodes, it captures key information. To further address the complex non-linear relationships in graph data, an RNN layer is introduced to enhance the expression of features, with the output of the final hidden layer serving as the features for the relational connectivity graph. The process of extracting RCG features $\boldsymbol{F_{RCG}}$ is shown in Fig. 4, step d.

3.5. Function-Level vulnerability detection

The ultimate goal of the vulnerability detection model is to output the detection results of the data. After obtaining the intra-function features $\boldsymbol{F_{PDG}}$ and inter-function features $\boldsymbol{F_{RCG}}$ in Sections 3.3 and 3.4, the model passes features into an MLP through comprehensive concatenation to process nonlinear complex data in the source code. After processing, the softmax function is used to calculate the probability distribution of each sample in each category to obtain the final vulnerability detection result. Specifically, the mathematical expression of the classifier network is as follows:

$$p\left(y|g^{PDG}\right) = \mathrm{softmax}(W_{CP}(Concat(\boldsymbol{F_{PDG}}, \boldsymbol{F_{RCG}})) + b_{CP}) \tag{14}$$

In this context, $p(y|g^{PDG})$ represents the actual output of the model, while $y \in \{0, 1\}$ represents the category labels of the samples. To optimize model performance, a cross-entropy loss function is used to measure the difference between the predicted results and the actual labels. The formula is shown as (15). In the formula, $Data \in \{TrainData, TestData\}$ represents the dataset for which the loss value needs to be calculated, and $q(y|g^{PDG})$ is the expected output of the model.

$$L = -\sum_{g^{PDG} \in Data} \sum_{y \in \{0,1\}} \log\left(p\left(y|g^{PDG}\right)\right) \cdot q\left(y|g^{PDG}\right) \tag{15}$$

To accurately locate vulnerabilities, this paper utilizes the token-level and node-level importance mechanisms of the VulTR model to construct a function vulnerability statement localization model, as detailed in the vulnerability localization stage shown in Fig. 1. First, the model obtains token attention scores w_{ij} for the target PDG node sequence through the token importance mechanism. By integrating the tokens of each statement through weighting, the attention score $\sum_{j=1}^{n} w_{ij}$ for each code statement is calculated. To reflect the importance of each node within the graph network, the statement importance is multiplied by the average centrality features C_i of the corresponding nodes in the PDG, thus deriving a comprehensive node importance score. Finally, the model ranks the nodes based on their importance; nodes with higher scores are considered crucial for interpreting the results of vulnerability

Line-level Vulnerability Localization by VulTR		Attention score	Centrality weight	VulTR
0	void FUN1(){	21.78	0.24	2.22
1	int VAR1;	16.27	0.43	7.00
2	VAR1=-1;	18.19	0.11	2.00
3	VAR1=FUN2(VAR1);{	35.37	0.04	1.41
4	int VAR2;	17.57	0.13	2.28
5	int VAR3[10]={0};	29.12	0.07	2.03
6	if(VAR1>=0){	32.50	0.25	8.13
7	VAR3[VAR1]=1;	54.64	0.49	26.77
8	for(VAR2=0;VAR2<10;VAR2++){	64.32	0.07	4.50
9	FUN3(VAR3[VAR2]);}}	26.02	0.99	25.76
10	else{FUN4("");}	37.17	0.32	11.89
11	}	13.78	0.07	0.96
12	}	4.62	0.02	0.09

Fig. 5. A motivating example of VulTR.

detection, indicating a higher correlation with the vulnerability. The calculation formula is as follows:

$$S_i = \left(\sum_{j=1}^{n} w_{ij} \right) \times C_i \tag{16}$$

Based on the above content, this study further applied the VulTR model for a practical case analysis, precisely locating vulnerabilities at the statement level within the FUN1 function. The model identified an array out-of-bounds access vulnerability in the FUN1 function, displayed in Fig. 5, located on line seven of the code. The VulTR model assigned the highest risk score to this vulnerability. The key node line numbers and their corresponding scores are: 7 (26.77), 9 (25.76), 10 (11.89), and 6 (8.13). This provides security personnel with a priority for vulnerability line detection and offers a degree of interpretability.

4. Experiments

This section designs the following questions to evaluate the effectiveness of the VulTR model:

RQ1: How does VulTR perform compared to state-of-the-art vulnerability detection methods?

RQ2: How is the centrality performance of different combinations in the node-level importance mechanism?

RQ3: How do the RCG and triple importance mechanism affect the model's performance?

RQ4: How effective is the model in locating vulnerable statements?

4.1. Dataset

This paper uses three datasets to verify the effectiveness of the experiment, including SARD+NVD (2021) , FFmpeg+Qemu (Zhou et al., 2019), and Big-Vul (Fan et al., 2020). The SARD+NVD dataset combines synthetic programs and real data, using functions written in C/C++ as the data source. Since most of the code functions in this dataset are synthetic data, they are not suitable for evaluating the detection of real vulnerabilities. This paper further compares two real datasets: the FFmpeg+Qemu dataset, which includes 24,524 real programs; and the Big-Vul dataset, which provides statement-level vulnerabilities and can help predict vulnerable lines, as shown in Table 2.

4.2. Experimental setup

This paper implements the proposed VulTR model by conducting experiments on a server equipped with 128GB RAM, 20-core CPU, and NVIDIA GeForce RTX 4090. First, according to the number of code functions in the data set, the data set is randomly divided into a training set, a verification set, and a test set in a ratio of 8:1:1. For function-level detection tasks, the PyTorch framework with powerful backpropagation and parameter optimization functions is selected as the computing tool. For vulnerability statement location, the Transformer model and architecture provided by (Feng et al., 2020) are used to improve location accuracy. During the model training process, the cross-entropy loss function and AdamW optimizer are used to update the model parameters. The training limit is set to 100 epochs, and when the performance does not improve, the early stopping strategy is adopted. The relevant parameters used in the experiment are shown in Table 3.

Table 2
Datasets.

Datasets	Total	Vul	No-vul
SARD+NVD	40,657	13,687	26,970
FFMPeg+Qemu	24,524	11,628	12,896
BigVul	19,206	8453	10,753

Table 3
Parameter settings.

Parameters	settings
loss function	Cross Entropy Loss
activation function	ReLU
optimizer	AdamW
batch size	32
learning rate	0.001
dropout	0.5

4.3. Metrics

To evaluate the performance of the model, common metrics were chosen to describe the effectiveness of the model:

- True Positive (TP): The number of samples that are correctly predicted to have vulnerable attributes.
- False Positive (FP): Number of samples incorrectly predicted to have vulnerable attributes.
- True Negative (TN): The number of samples correctly predicted as normal attributes.
- False Negative (FN): The number of samples incorrectly predicted as normal attributes.
- Accuracy: The proportion of instances that are correctly labeled out of all instances.
- Recall: Proportion of correctly predicted positive examples among actual vulnerable samples.

$$Recall = \frac{TP}{TP + FN} \tag{17}$$

- Precision:Proportion of correct predictions among samples predicted to be vulnerable

$$Precision = \frac{TP}{TP + FP} \tag{18}$$

- *F*1:It balances the relationship between Precision and Recall, serving as a comprehensive evaluation index for measuring the two-class classification model.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{19}$$

4.4. Results for RQ1

This paper compares the following models to explore the advantages of the multi-level feature enhancement of the VulTR model:

When studying question RQ1, we first use the SARD+NVD dataset to compare it with five baseline methods, including two code slicing-based text models and three function-based graph models. The relevant indicators and results are shown in Fig. 6.

Considering that the SARD+NVD dataset is synthetic, to better reflect the complexity of the actual programming environment, two real datasets, FFmpeg+Qemu and BigVul, are introduced to further verify the effectiveness of the model in real applications. The corresponding analysis results are detailed in Table 4.

For graph models, Devign uses GGNN to achieve neighborhood aggregation and learn node dependencies by aggregating the information of adjacent nodes. However, since it mainly focuses on local connections and treats all nodes equally, this approach limits the distinction between hierarchical structures and structures of the code, which may affect the accurate identification of vulnerable code. IVDetect (Y Li et al., 2021) integrates multiple features, including AST, data dependency, control dependency, and context sequence. However, in the process of node embedding, the token is simply passed into the linear layer to obtain the

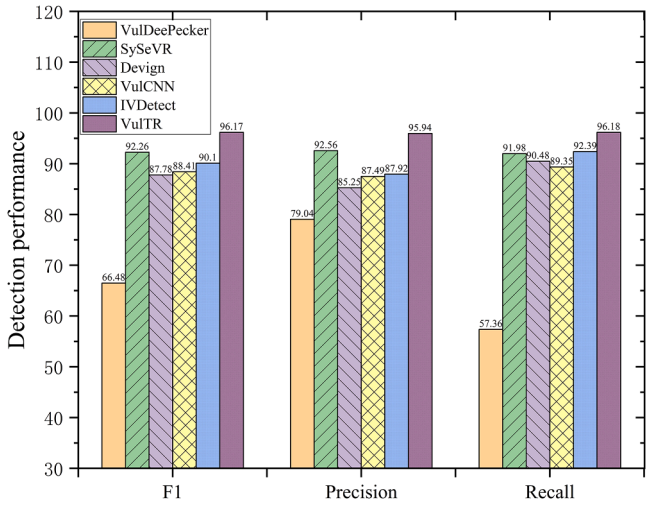

Fig. 6. Performance comparison of the VulTR model with other models on the SARD+NVD dataset.

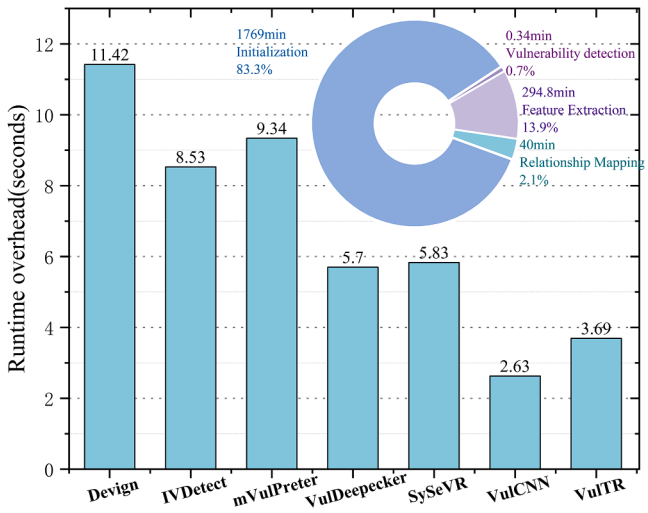

Fig. 7. Runtime analysis.

Table 4
A comparison of performance between the VulTR model and other models at reall word datasets.

	FFMPeg+Qemu			BigVul		
	Precision	Recall	F1	Precision	Recall	F1
VulDeePecker (Li et al., 2018)	48.64	49.96	49.29	44.82	40.36	42.47
SySeVR (Zhen et al., 2021)	45.01	57.85	50.63	48.93	59.34	53.63
Devign (Zhou et al., 2019)	52.66	51.16	51.89	40.32	41.27	40.79
VulCNN (Wu et al., 2022)	45.24	42.53	43.84	42.36	34.15	37.81
IVDetect (Y Li et al., 2021)	50.19	60.34	54.79	34.26	58.37	43.17
mVulPreter (Zou et al., 2022)	——	——	——	51.56	59.38	55.19
VulTR	54.81	55.53	55.17	60.52	61.28	60.90

initial node representation, ignoring the dependencies between tokens in the statement node. The mVulPreter model combines function-level and slice-level features to improve the accuracy of vulnerability detection. However, it relies on clear vulnerability line numbers in the dataset to ensure the accuracy of slice annotation. Since the SARD+NVD and FFmpeg+Qemu datasets lack this information, the model is only evaluated on the BigVul dataset, which has rich patch information. Although it integrates key features, it does not fully consider the relationship between functions, resulting in performance that is inferior to VulTR. For VulCNN, it simply converts code information into a vector combined with centrality and uses CNN to extract model features, resulting in the worst vulnerability detection effect.

The VulTR model performs well on three different datasets. Specifically, compared to the optimal baseline method, the F1 score of the model on these three datasets is improved by 3.91 %, 0.38 %, and 5.71 %, respectively. This performance improvement reflects the effectiveness of the model's vulnerability detection.

To gain a deeper understanding of the efficiency of the model, this study further evaluated the time performance of the VulTR model on synthetic datasets, as shown in Fig. 7. The whole process is divided into four main stages: the initialization stage, the feature extraction stage, the relationship mapping stage, and the vulnerability identification stage. Since the mVulPreter model is only implemented on the BigVul dataset, this section performs a time performance analysis on the BigVul dataset to clarify the performance of different models in terms of processing time. In Fig. 7, the pie chart shows the time distribution of VulTR on BigVul, where PDG consumes more time; the bar chart compares the running times of various models on BigVul. For VulDeePecker and SySeVR, the slice generation stage involves extracting CFG, PDG, and FCG. Since only data flow or control flow information is needed, feature fusion is minimal, resulting in a smaller time overhead compared to VulTR. For IVDetect, which needs to extract AST, data dependency, and control dependency sequences as code features, it takes more time and is less efficient in large-scale detection. The Devign model combines multiple code representations, including AST, CFG, DFG, and natural code sequences (NSC), to achieve accurate vulnerability analysis. Due to the extensive amount of code representation it extracts, Devign's running time exceeds that of VulTR. Although its scalability is not as robust as VulCNN, the detection performance of VulTR demonstrates a clear advantage, proving the effectiveness of the time investment.

4.5. Results for RQ2

This section discusses the impact of different combinations of centrality indicators on the model. Currently, social network centrality is mainly divided into five cases. For clarity, it is summarized into three categories according to the core calculation concept: centrality based on core nodes, centrality based on social relationships between nodes, and centrality based on the distance between nodes. Section 3.3.2 introduces the concepts and calculation methods of degree centrality, eigenvector centrality, and betweenness centrality. This section further explores the content of the other two centrality indicators.

Katz centrality extends eigenvector centrality by considering both direct connections and the influence of indirect neighbors. It adjusts the importance of distant neighbors using the α parameter and reflects the node's inherent importance through the β parameter.

$$C_K(v_i) = \alpha \sum_{u \in V} A_{uv_i} C_K(u) + \beta \tag{20}$$

Distance-based closeness centrality quantifies the average proximity of the current node to other nodes by calculating the sum of the shortest paths $d(v_i, u)$ on the node's potential paths.

$$C_C(v_i) = \frac{|V| - 1}{\sum_{u \in V \setminus \{v_i\}} d(v_i, u)} \tag{21}$$

In order to explore the impact of different indicator combinations on feature capture, the experiment converts the three features of each combination into image information and evaluates them through an intensity-frequency histogram. The images of the four combinations on the three datasets are similar. It can be observed from Fig. 8 that

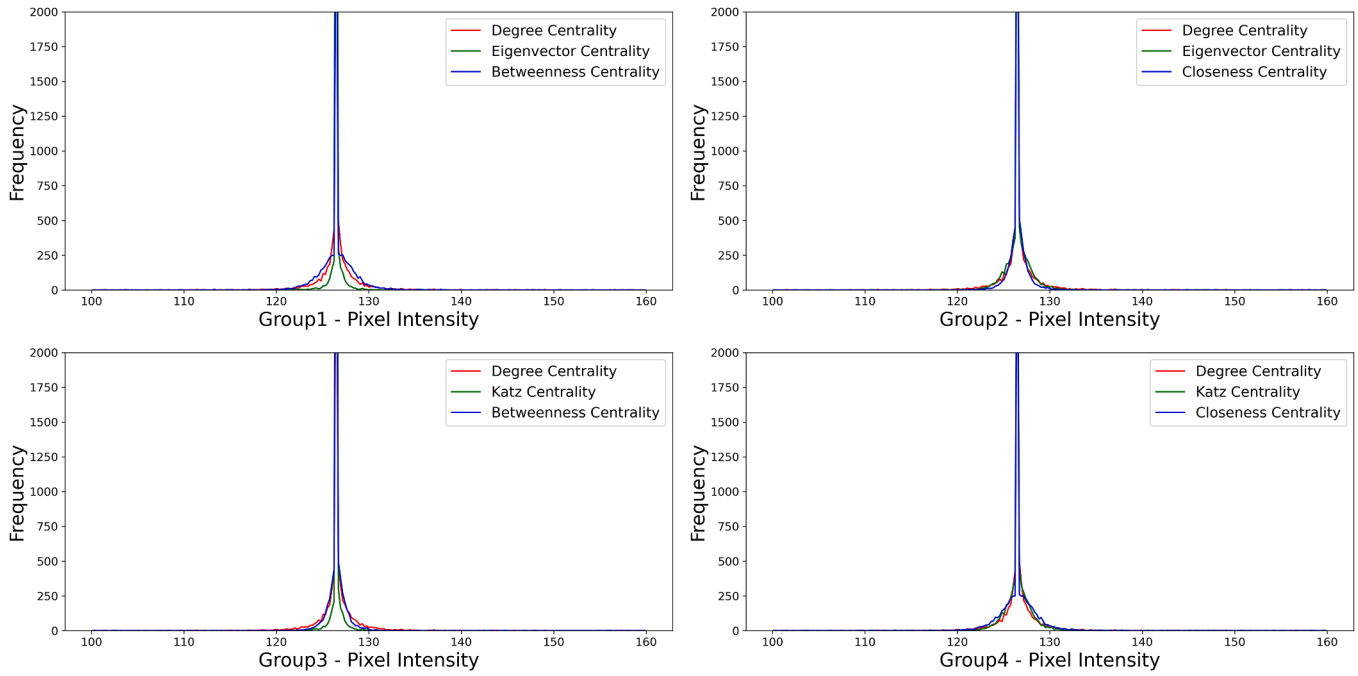

Fig. 8. Comparison of model performance across different centrality combinations.

combinations 2 and 4 may not capture different aspects of the data due to the overlap of information from closely related centralities with other indicators, resulting in duplication and redundancy. The frequencies of different centrality indicators in combination 3 differ, making it more likely to capture diverse characteristics of the data. Compared to other combinations, the three features of combination 1 have a relatively clear distinction and a wider frequency distribution, showing more diversity in centrality measurement. The performance data of different combinations on the three datasets is shown in Table 5.

According to the analysis of the histogram and Table 5, different combinations of centrality indicators have a multidimensional impact on classification performance; multiple similar centrality indicators may reduce the model's ability to distinguish, thereby limiting performance improvement. However, in the two real datasets, the centrality gap between different combinations is not large, with the maximum differences in the three datasets being 16.99, 5.94, and 8.24, respectively. This may be because the features of synthetic datasets are often more obvious and prominent, as the dataset design may emphasize specific vulnerability characteristics to facilitate model learning and testing. The features of real datasets are usually more hidden and complex, reflecting the diversity and uncertainty of real-world vulnerabilities.

Given the impact of different centrality features, it is crucial to explore various integration methods to take advantage of their joint benefits in network models. When fusing three different types of centrality features in the network, a variety of methods and strategies can be employed. Specifically, these include: direct concatenation, i.e., directly concatenating the three centrality feature vectors according to their vector dimensions; calculating the mean vector, achieved by averaging the centrality vectors along their tensor dimensions; and using a convolutional model to extract and fuse the three-channel information to obtain the node vector. Fig. 9 illustrates the differences between the proposed feature acquisition method and the above three strategies, indicating that the VulTR model can more effectively fuse node features.

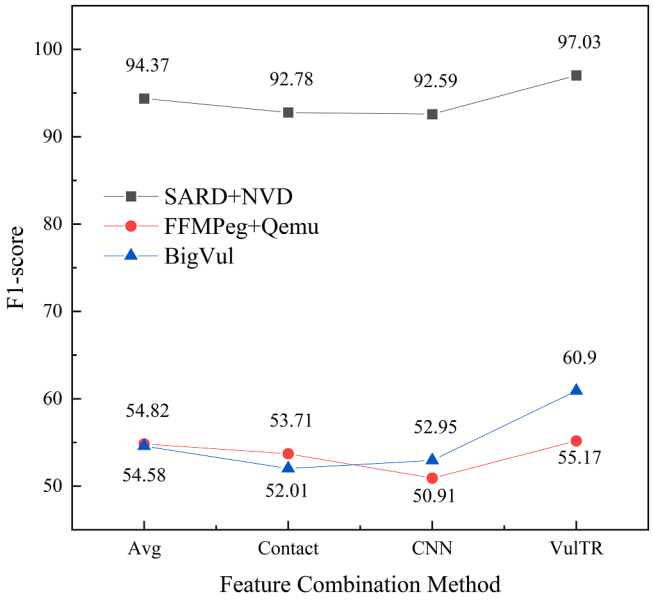

Fig. 9. Performance comparison of different feature fusion strategies.

Table 5
Comparing the performance of centrality metrics across various datasets.

Dataset	Group1		Group2		Group3		Group4	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1
SARD+NVD	95.71	96.06	83.56	79.07	91.23	82.88	85.71	80.40
FFMPeg+Qemu	64.36	60.90	62.94	54.96	63.03	57.52	61.38	55.23
BigVul	63.00	55.17	61.94	46.93	63.72	51.86	58.27	49.06

4.6. Results for RQ3

In order to fully consider the syntax and semantic information of the code, VulTR uses the k-GCNs model with a triple attention mechanism to obtain intra-function features and fuse them with the relationship connection graph features. To deeply study their impact on model performance, ablation experiments are performed in this section to explore the effectiveness of each part of the features. Two modules were specifically studied, including the multi-importance graph convolutional network module (TRI) and the functional relationship connection graph representation module (RCG).

TRI module: This module integrates token-level attention, node-level centrality, and graph-level attention mechanisms. In order to prove the effectiveness of the TRI module, this study designed a series of ablation experiments, including VulTR w/o-token, VulTR w/o-node, and VulTR w/o-graph. These experiments remove token-level, node-level, and graph-level importance mechanisms respectively, and use word2vec, MLP, and Conv as baseline models for comparison, thereby systematically evaluating the specific contribution of each importance mechanism in parsing PDG vulnerability patterns.

Experimental results show that the triple importance mechanism significantly improves graph classification performance compared to the baseline model. First, removing the graph-level importance mechanism has the greatest impact on classification results. The graph-level importance mechanism shows significant differences from the traditional convolution method in handling graph-level importance. It can process the relationship between nodes layer by layer in a cascading manner and propagate information between different subgraphs of the graph. In contrast, traditional convolutional methods mainly rely on local receptive fields and are weak in processing global structural information. In addition, the graph-level importance mechanism can automatically learn the importance of nodes and edges in the graph to more accurately identify and analyze the global characteristics of the code.

Secondly, in the PDG node embedding process, a token-level importance mechanism is adopted. The traditional Word2Vec model can effectively capture the relationship between words (such as semantic similarity), but because it is mainly designed for natural language text, it does not adequately capture the structural patterns and grammatical dependencies in the code. In contrast, the token-based importance mechanism uses a pre-trained model to consider the specific purpose, meaning, and grammatical structure of the token, improving the understanding of the overall code logic.

At the node level, although MLP is often used to describe interactions, it fails to effectively consider the role of nodes and their respective importance. This uniform processing method may cause the characteristics of key nodes to be submerged, affecting the final analysis effect. On the contrary, the centrality measure used in this paper can effectively distinguish and emphasize the importance of the same node not only by expressing the graph structure information but also by giving each node an appropriate weight according to its position and role in the network, thereby assisting the graph-level model to obtain better analysis results.

In summary, the experimental results indicate that accurate identification of vulnerability patterns requires comprehensive consideration of the semantics, structure, and contextual information of the vulnerable code. The TRI module achieves higher vulnerability detection capabilities by strengthening the focus on key elements, in which token-level, node-level, and graph-level importance mechanisms are all effective. Compared with the other two hierarchical mechanisms, the contribution of node-level centrality is smaller, probably because it mainly focuses on local structure and has limited ability to capture global information.

2) RCG module: To understand the performance of the functional relationship connection graph representation module, the model variant VulTR w/o RCG was experimentally deployed. As can be seen from Table 6, the F1 score of VulTR has increased by 6.40 %, 7.04 %, and 12.63 % respectively on the SARD, FFmpeg+Qemu, and BigVul datasets; the accuracy of VulTR has increased by 4.35 %, 3.18 %, and 9.45 % respectively. Experimental results show that the RCG module improves the performance of vulnerability detection significantly.

3) VulTR model overfitting analysis and performance evaluation: By combining the two modules, VulTR has improved on all three datasets. This demonstrates that strengthening the focus on key elements and combining the dependency characteristics between functions play a key role in enhancing the comprehensive performance of VulTR. To avoid model overfitting due to noise and random variations in the training dataset, the model employs a dropout rate of 0.5 to enhance its generalization ability to new data. Fig. 10 depicts the evolution of the loss value of the VulTR model during the prediction process, with the horizontal axis representing the number of training iterations. Using the synthetic dataset as an example, the loss value drops rapidly in the first 50 iterations of training, indicating that the model has high learning efficiency in the early stages and is quickly learning important features from the training data. After 50 iterations, the rate of loss decline begins to slow and stabilizes at around 200 iterations, eventually stabilizing at approximately 0.30. This suggests that the model may have approached its learning limit. Throughout the training process, the training loss closely matches the validation loss, indicating that the model is not overfitting. This confirms that the VulTR model can effectively learn and improve prediction accuracy. However, on the FFMPeg+Qemu and BigVul datasets, the model's performance is not as good, indicating a higher likelihood of overfitting on real datasets. Future research should focus on solving this problem.

4.7. Results for RQ4

This study designed a vulnerability statement localization model that leverages the token-level and node-level importance mechanisms of the VulTR model, and compared it with the accuracy of the mVulPreter model. According to the standards of the literature (Y Li et al., 2021), if the model's explanation overlaps with the code changes that fix the vulnerability, it is considered accurate. The accuracy is calculated based on the proportion of correct explanations. Compared to previous external model-agnostic technologies (Wattanakriengkrai et al., 2020; Pornprasit et al., 2021), mVulPreter and VulTR are built-in model-agnostic technologies (i.e., the DL/ML model itself is interpretable and does not require additional models). They are inherently interpretable and do not require additional technology or complex training. The importance of each sentence can be evaluated through straightforward calculations.

Table 6
The effectiveness of different key element importances and the RCG module.

Model	SARD+NVD			FFMPeg+Qemu			BigVul		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
VulTR w/o-token	90.96	89.87	90.41	53.07	48.94	50.92	51.94	50.62	51.27
VulTR w/o-node	90.96	92.87	91.90	49.52	53.89	51.61	50.96	54.83	52.82
VulTR w/o-graph	91.25	87.48	89.32	51.06	46.15	48.48	52.65	49.36	50.95
VulTR w/o-RCG	91.94	88.63	90.25	53.12	50.06	51.54	55.29	50.91	54.07
VulTR	95.94	96.18	96.06	54.81	55.53	55.17	60.52	61.28	60.90

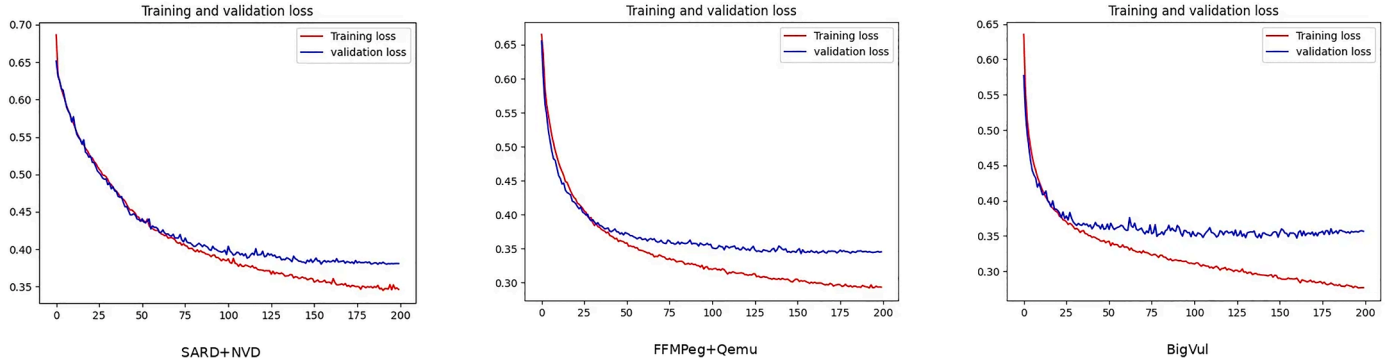

Fig. 10. Analysis of model overfitting.

Table 7
Performance of vulnerability statement localization.

Model	Accuracy
mVulPreter	0.58
VulTR	0.64

This section conducts experiments on a subset of Big-Vul, which includes vulnerability patches and corresponding functions before and after the patches. This setup better facilitates research on line-level vulnerability localization. The results in Table 7 show that the VulTR model outperforms mVulPreter in terms of accuracy, reaching 64 %. This advantage mainly stems from the meticulous approach adopted by VulTR in vulnerability detection, including its unique token-level and node-level importance mechanisms. The application of these technologies not only improves the accuracy of vulnerability detection but also enhances the model's ability to understand complex code structures. Therefore, VulTR's method provides a more effective solution for line-level defect prediction, which has significant practical implications for improving software security.

5. Discussion

Internal Threats: These are related to the selection of hyperparameters within the model. To prevent the VulTR model from overfitting, adjustments to the model parameters were made, and overfitting analysis experiments were conducted, assuming a sufficient dataset.

External Threats: These are related to the quality of the datasets. Since SARD+NVD contain synthetic data that may not fully represent real-world scenarios, this study introduced two real datasets, BigVul and FFMPeg+Qemu, to further validate the model's vulnerability detection effectiveness.

Limitations and Future Directions: As software development continues to evolve, vulnerability detection models (including models such as IVDetect and mVulPreter) must overcome the limitations of being confined to a single programming language. Although VulTR has demonstrated excellent performance in function-level vulnerability detection for C/C++ languages, the significant differences in syntax and structure across various programming languages necessitate further validation of its adaptability to other languages. While VulCNN theoretically offers cross-language generalizability, its actual detection performance still needs improvement. Therefore, future work should focus on developing universal models that can operate across multiple programming languages.

Additionally, although VulTR performs well on synthetic datasets (e.g., SARD+NVD), its performance in handling more complex real-world vulnerabilities (e.g., BigVul) during experiments (RQ1) has been less satisfactory. Future research could consider employing contrastive learning techniques, leveraging data augmentation to enrich real-world training data, thereby enhancing the model's ability to distinguish between vulnerable and non-vulnerable code. To further improve the practical utility of vulnerability detection models, future studies could explore integrating VulTR with ChatGPT. In this approach, the VulTR model would be responsible for accurately locating vulnerabilities within the code, while the ChatGPT model would generate potential repair code based on the detected vulnerabilities. Throughout this process, it is crucial to ensure that the generated repair code meets practical requirements and does not introduce new vulnerabilities.

6. Conclusion

This paper introduces a new vulnerability detection framework named VulTR, which significantly enhances the focus on and feature capture of key source code elements by strengthening various elements within the PDG. Firstly, VulTR employs an improved embedding method to mitigate the sub-token OOV problem; at the node level, it enhances the importance of nodes in the PDG through centrality analysis; at the graph level, it uses an improved k-GCNs model to extract graph vector features. Additionally, VulTR constructs a function relation join graph, addressing the issue that existing models overlook relational space semantic features, thereby more effectively simulating features between functions. Experimental results show that compared to slice-based and function-level models, VulTR has achieved a significant improvement in F1 scores, ranging from 0.69 % to 61.07 %, on three datasets, substantially enhancing vulnerability detection performance.

CRediT authorship contribution statement

Haitao He: Writing – review & editing, Supervision, Methodology, Conceptualization. **Sheng Wang:** Writing – review & editing, Software, Formal analysis, Data curation. **Yanmin Wang:** Visualization, Investigation. **Ke Liu:** Validation, Software. **Lu Yu:** Resources, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was supported by National Natural Science Foundation of

China (no.62376240), S&T Program of Hebei (nos.226Z0701G, 236Z0702G, 236Z0304G), the Natural Science Foundation of Hebei Province (nos.F2022203026, F2022203089), Science Research Project of Hebei Education Department (no.BJK2022029) and Innovation Capability Improvement Plan Project of Hebei Province (no. 22567637H). The authors are grateful to the valuable comments and suggestions of the reviewers.

References

"Cyber security vulnerability statistics and facts of 2022.". 2022a, https://www.comparitech.com/blog/information-security/cybersecurity-vulnerability-statistics/.
"2022 open source security and risk analysis report.". 2022b, https://www.synopsys.com/software-integrity/resources/analyst-reports/open-source-security-risk-analysis.html.
Clang static analyzer. 2020a, https://clang-analyzer.llvm.org/scan-build.html.
Flawfinder. 2020b, https://dwheeler.com/flawfinder/.
Checkmarx. 2020c, https://www.checkmarx.com/.
CWE_121. 2024, "https://cwe.mitre.org/data/definitions/121.html".
Software Assurance Reference Dataset. 2021, https://samate.nist.gov/SRD/index.php.
Alon U., Yahav E. 2020. On the bottleneck of graph neural networks and its practical implications. arXiv preprint arXiv:2006.05205.
Ayewah, N., Pugh, W., Morgenthaler, J.D., Penix, J., Zhou, Y., 2007. Evaluating static analysis defect warnings on production software. In: Proceedings of the 7th ACM SIGPLAN-SIGSOFT workshop on Program analysis for software tools and engineering, pp. 1–8.
Cangea C U A T, Veli V.C. Kovi C.P., Jovanovi C.N., Kipf T., Li O.P. 2018. Towards sparse hierarchical graph classifiers. arXiv preprint arXiv:1811.01287.
Cao, S., Sun, X., Bo, L., Wei, Y., Li, B., 2021. Bgnn4vd: constructing bidirectional graph neural-network for vulnerability detection. Inf. Softw. Technol. 136, 106576.
Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2021. Deep learning based vulnerability detection: are we there yet. IEEE Trans. Software Eng.
Devlin J., Chang M., Lee K., Toutanova K. 2018. Bert: pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.
Duan, X., Wu, J., Ji, S., Rui, Z., Luo, T., Yang, M., Wu, Y, 2019. VulSniper: focus Your Attention to Shoot Fine-Grained Vulnerabilities. In: IJCAI, pp. 4665–4671.
Fan, J., Li, Y., Wang, S., Nguyen, T.N., 2020. AC/C++ code vulnerability dataset with code changes and CVE summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories, pp. 508–512.
Feng Z., Guo D., Tang D., Duan N., Feng X., Gong M., Shou L., Qin B., Liu T., Jiang D., Others. 2020. Codebert: a pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155.
H F., X F., X S. Efficient Vulnerability Detection based on abstract syntax tree and Deep Learning.2020.
Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. Adv. Neural. Inf. Process Syst 30.
Hellendoorn, V.J., Sutton, C., Singh, R., Maniatis, P., Bieber, D., 2019. Global relational models of source code. In: International conference on learning representations.
Hochreiter, S., Schmidhuber, J.U.R., 1997. Long short-term memory. Neural. Comput. 9 (8), 1735–1780.
Hu, Y., Zou, D., Peng, J., Wu, Y., Shan, J., Jin, H., 2022. TreeCen: building tree graph for scalable semantic code clone detection. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–12.
Jang J., Agrawal A.D. 2012. Redebug: finding unpatched code clones in entire os distributions 48-62.
Johnson, B., Song, Y., Murphy-Hill, E., Bowdidge, R., 2013. Why don't software developers use static analysis tools to find bugs?. In: 2013 35th International Conference on Software Engineering (ICSE). IEEE, pp. 672–681.
Kim S., Woo S., Lee H. 2017. Vuddy: a scalable approach for vulnerable code clone discovery 595-614.
Lecun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. Nature 521 (7553), 436–444.
Li Q., Han Z., Wu X. Deeper insights into graph convolutional networks for semi-supervised learning.2018a.
Li, Y., Gu, C., Dullien, T., Vinyals, O., Kohli, P., 2019. Graph matching networks for learning the similarity of graph structured objects. In: International conference on machine learning. PMLR, pp. 3835–3845.
Li, Y., Wang, S., Nguyen, T.N., 2021a. Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 292–303.
Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., Jin, H., 2021b. Vuldeelocator: a deep learning-based fine-grained vulnerability detector. IEEE Trans. Dependable Secure Comput. 19 (4), 2821–2837.
Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y, 2018. VulDeePecker: a deep learning-based system for vulnerability detection. National Down Syndrome Society.
Liu, B., Shi, L., Cai, Z., Li, M., 2012. Software vulnerability discovery techniques: a survey. In: 2012 fourth international conference on multimedia information networking and security. IEEE, pp. 152–156.
Lloyd, S., 1982. Least squares quantization in PCM. IEEE Trans. Inf. Theory 28 (2), 129–137.
Lomio, F., Iannone, E., De Lucia, A., Palomba, F., Lenarduzzi, V., 2022. Just-in-time software vulnerability detection: are we there yet? J. Syst. Software 188, 111283.
Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J., 2013. Distributed representations of words and phrases and their compositionality. Adv. Neural. Inf. Process Syst 26.
Neamtiu, I., Foster, J.S., Hicks, M., 2005. Understanding source code evolution using abstract syntax tree matching. In: Proceedings of the 2005 international workshop on Mining software repositories, pp. 1–5.
Newsome, J., Song, D.X., 2005. Dynamic taint analysis for automatic detection, analysis, and signaturegeneration of exploits on commodity software. In: NDSS. Citeseer, pp. 3–4.
Okun, V., Delaitre, A., Black, P.E., 2013. Report on the static analysis tool exposition (sate) iv. NIST Spec. Publ. 500, 297.
Pham, Nam H and Nguyen, Tung Thanh and Nguyen, Hoan Anh and Nguyen, Tien NNam H. Pham T T N H. 2010. Detection of recurring software vulnerabilities. Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering. 447-456.
Pagliardini M., Gupta P., Jaggi M. 2017. Unsupervised learning of sentence embeddings using compositional n-gram features. arXiv preprint arXiv:1703.02507.
Pei H., Wei B., Chang K.C., Lei Y., Yang B. 2020. Geom-gcn: geometric graph convolutional networks. arXiv preprint arXiv:2002.05287.
Pornprasit, C., Tantithamthavorn, C.K, 2021. In: Jitline: A simpler, better, faster, finer-grained just-in-time defect prediction. IEEE.
Russell R.L., Kim L., Hamilton L.H., Lazovich T., Harer J.A., Ozdemir O., Ellingwood P. M., Mcconley M.W. 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning 757-762.
Sennrich R., Haddow B., Birch A. 2015. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909.
Smith J., Johnson B., Murphy-Hill E., Chu B., Lipford H.R. Questions developers ask while diagnosing potential security vulnerabilities with static analysis.2015.
Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Li, O.P, 2018. Yoshua 391 Bengio. Graph attention networks. In: International Conference on Learning Representations, p. 393.
Wattanakriengkrai, S., Thongtanunam, P., Tantithamthavorn, C., Hata, H., Matsumoto, K., 2020. Predicting defective lines using a model-agnostic technique. IEEE Trans. Software Eng. 48 (5), 1480–1496.
Wu Y., Zou D., Dou S., Yang W., Xu D., Jin H. 2022. VulCNN: an Image-inspired Scalable Vulnerability Detection System ACM: 2365-2376.
Xiao, C., Haoyu, W., Jiayi, H., Guoai, X., Yulei, S., 2021. DeepWukong statically detecting software vulnerabilities. ACM Transactions on Software Engineering and Methodology 30 (3), 1–33.
Xu K., Hu W., Leskovec J., Jegelka S. 2018. How powerful are graph neural networks?. arXiv preprint arXiv:1810.00826.
Y, H., S, W., Y, W, 2023. A Slice-level vulnerability detection and interpretation method based on graph neural network. J. Software 34 (6).
Yamaguchi, F., Golde, N., Arp, D., Rieck, K., 2014. Modeling and discovering vulnerabilities with code property graphs. In: 2014 IEEE Symposium on Security and Privacy. IEEE, pp. 590–604.
Zhen, L., Deqing, Z., Shouhuai, X., Hai, J., Yawei, Z., Zhaoxuan, C., 2021. SySeVR: a framework for using deep learning to detect software vulnerabilities. IEEE Trans. Dependable Secure Comput.
Zhou, Y., Shangqing, Jing L, Siow, K, 2019. Devign: effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. NeurIPS, 2019.
Zhu, D., Dai, X., Chen, J., 2021. Pre-train and learn: preserving global information for graph neural networks. J. Comput. Sci. Technol. 36, 1420–1430.
Zou, D., Hu, Y., Li, W., Wu, Y., Zhao, H., Jin, H., 2022. mVulPreter: a multi-granularity vulnerability detection system with interpretations. IEEE Trans. Dependable Secure Comput.
Zou, D., Wang, S., Xu, S., Li, Z., Jin, H., 2019. μVulDeePecker: a deep learning-based system for multiclass vulnerability detection. IEEE Trans. Dependable Secure Comput. 18 (5), 2224–2236.

Haitao He is a professor at the School of Information Science and Engineering, Yanshan University. Her research interests include AI, Data mining, Software security analysis.

Sheng Wang, born in 1999. He received the B.S. degree from the School of Artificial Intelligence, Tangshan University, China, in 2022. He is currently a master's degree at the School of Information Science and Engineering, Yanshan University. His research interests are software security, Data mining, and AI.

Ke Liu, born in 1998. He received the B.E. degree from the School of Information and Computer Science, Taiyuan University of Technology, China, in 2020. He is currently a master's degree at the School of Information Science and Engineering, Yanshan University. His research interests are network security, Poisoning attack defense, and AI.

Yanmin Wang, born in 1998. She received the B.S. degree from the Department of Computer Science and Technology, Tangshan University, China, in 2021. She is currently a master's degree at the School of Information Science and Engineering, Yanshan University. Her research interests inculde deep learning and software security.

Lu Yu, born in 1987. She received her master's degree in 2015 from the School of Information Science and Engineering, Yanshan University, China. She is currently a senior engineer at the Hebei Port Group CO., LTD. Her research interests include network security and data mining.